AI System for Procedural Texture Generation from Real-World Photographs

Research and Development Project Proposal for Cloud Gaming and Data Center Infrastructure

Author: Milan Majstorović

Date: October 19, 2026

Table of Contents

Abstract 4

1. Introduction 4

2. Subject, Problem, and Motivation 5

   2.1 Research Question 5

   2.2 Working Hypotheses 5

3. Background and Related Work 5

   3.1 Conventional and Neural Texture Compression 5

   3.2 Procedural Textures 6

   3.3 Program Synthesis and Differentiable Rendering 6

   3.4 Research Gap 6

4. Project Objectives and Contribution 6

   4.1 Original Contribution 7

5. Proposed System Architecture 7

   5.1 Input Preparation 7

   5.2 Visual Encoder 7

   5.3 Procedural Graph Generator 7

   5.4 Parameter Optimizer 8

   5.5 Hybrid Residual Layer 8

   5.6 Shader Code Generator and Verifier 8

6. Research Methodology 8

   6.1 Dataset 8

   6.2 Baseline Methods 8

   6.3 Objective Function 9

   6.4 Metrics 10

   6.5 Hardware and Software Environment 10

   6.6 Experimental Procedure 10

   6.7 Statistical Analysis and Reproducibility 11

   6.8 Success Criteria 11

7. Application to Cloud Gaming 11
7.1 Potential Benefits 11
7.2 Important Limitation 12
7.3 Adaptive Policy 12
8. Broader Implications for Data Center Infrastructure 12
8.1 Storage, Replication, and Caching 12
8.2 Computational Task Scheduling 12
8.3 Energy and Carbon Efficiency 12
8.4 Applications Beyond Gaming 13
8.5 Future Directions: Photonic and Hybrid Architectures 13
9. Risks and Limitations 14
10. Ethical, Legal, and Environmental Considerations 15
10.1 Data, Copyright, and Licenses 15
10.2 Validation and Security of AI-Generated Code 15
10.3 Transparency and Reproducibility 15
10.4 Environmental Assessment 15
11. Research and Development Plan 16
12. Expected Results 16
13. Conclusion 16
14. References 17

Abstract

This research and development project proposal presents a system that uses artificial intelligence to transform real-world photographs into compact procedural representations and executable *shader* code. The goal is not merely to reduce the size of texture assets, but to optimize the overall trade-off among memory, storage, network transfer, loading time, and execution cost. Cloud gaming was selected as the primary application domain because it enables centralized resource management and the measurement of effects across a large number of instances. The broader implications for data centers are considered in a separate section, as they encompass workload scheduling, caching, data replication, and energy efficiency. The proposed methodology includes dataset construction, procedural program synthesis, shader code generation, multi-objective optimization, and experimental comparison with conventional and neural approaches.

Keywords: procedural textures, artificial intelligence, program synthesis, shader, compression, cloud gaming, data centers.

1. Introduction

Modern interactive applications use large numbers of high-resolution textures, increasing storage, memory, and bandwidth requirements. Conventional block-compression formats are optimized for rapid decoding on the graphics processing unit (GPU), while neural methods seek to reduce the footprint through on-demand decoding [1], [9]. A procedural representation uses mathematical functions and a small set of parameters to reconstruct content at runtime.

Procedural shading and noise functions have long been used to create visual complexity compactly [4]–[6]. Designing them manually, however, requires expert knowledge, while the additional computation may increase CPU and GPU load. Recent research shows that procedural material graphs can be generated and optimized automatically from an image or text [2], [3].

In this work, cloud gaming is not an afterthought but one of the primary evaluation scenarios. Cloud platforms centralize rendering and resource management, yet computing power still affects cost, energy consumption, user density, and latency [7], [8]. The project therefore treats procedural generation as a measurable trade-off between data size and execution cost rather than as an unconditional replacement for conventional textures.

2. Subject, Problem, and Motivation

The research subject is the automatic discovery of a procedural program that visually approximates an input photograph or texture sample. The program must be sufficiently compact, portable, and efficient to execute in the selected graphics environment.

The central problem can be formulated as a multi-objective optimization task: perceptual similarity to the original must be increased while program size, generation time, execution time, memory consumption, and the amount of stored or transferred data are reduced. These objectives may conflict, so the system must identify a set of trade-off solutions rather than a single universal optimum.

2.1 Research Question

Can an AI system automatically generate a procedural representation of a real-world texture that, at an acceptable level of visual quality, reduces total infrastructure cost compared with conventional texture storage and delivery?

2.2 Working Hypotheses

1. For textures with pronounced stochastic, periodic, or multiscale structure, a smaller representation can be achieved with acceptable perceptual loss.
2. A hybrid approach that combines procedural functions with small residual textures will be more robust than a fully procedural approach for complex photographs.
3. Storage and I/O gains will not automatically result in a lower total cost; the outcome will depend on reuse frequency, caching, and CPU/GPU execution cost.
4. In a cloud gaming environment, adaptive selection among conventional, procedural, and hybrid representations can improve infrastructure utilization.

3. Background and Related Work

3.1 Conventional and Neural Texture Compression

Conventional compression formats store discrete blocks or transformed image coefficients and are optimized for rapid decoding. Neural methods learn a latent representation and use a small decoder for reconstruction; one example is random-access neural texture compression (NTC) with real-time decoding [1]. Both groups can achieve high quality, but they do not necessarily produce a readable procedural program or enable straightforward semantic modification of the texture structure.

3.2 Procedural Textures

Procedural textures are created by combining gradients, noise functions, fractal compositions, cellular structures, coordinate transformations, filtering, and layer blending. Perlin, wavelet, and Gabor approaches illustrate different trade-offs among speed, spectral control, and artifacts [4]–[6]. Advantages of procedural representations include compactness, resolution scalability, and parameterization, while disadvantages include execution cost and a limited ability to reproduce semantically complex photographs.

3.3 Program Synthesis and Differentiable Rendering

The problem can be viewed as program synthesis: the system selects an operator graph and its parameters so that the rendered output matches the target image. Existing studies demonstrate procedural graph generation from an image or text and gradient-based optimization of graphs containing non-differentiable operators [2], [3]. This project extends that direction with infrastructure objectives and export to executable *shader* code.

3.4 Research Gap

Within the limited set of reviewed sources, the existing literature separately addresses neural texture compression [1], procedural graph generation and optimization [2], [3], procedural noise functions [4]–[6], and cloud gaming infrastructure [7]. Based on this preliminary review, no unified system was identified that generates a portable graphics shader program from a real-world photograph, supports a hybrid residual layer, and optimizes the solution for quality, size, execution time, memory, I/O load, and energy. This combination represents a potential research gap targeted by the project; any claim of originality can be confirmed only after a documented systematic literature review.

4. Project Objectives and Contribution

1. Define a library of procedural operators and a constrained program grammar.
2. Develop a model for analyzing visual patterns and proposing the structure of a procedural graph.
3. Generate valid GLSL, HLSL, or WGSL code from the selected representation.
4. Optimize quality, size, execution time, and infrastructure cost.
5. Develop a hybrid representation for image regions that cannot be described efficiently through procedural methods.
6. Experimentally evaluate benefits in local and cloud gaming environments.
7. Formulate guidelines for broader application in data centers.

4.1 Original Contribution

The proposed contribution is neither a new individual noise function nor another neural codec. Its originality is based on integrating four elements:

1. **Infrastructure-aware program synthesis:** the procedural graph is optimized not only against the image but also against measurable execution and delivery costs.
2. **Adaptive three-way representation:** the system selects a conventional, fully procedural, or hybrid procedural-residual variant for each asset.
3. **Pareto output:** instead of a single solution, the system produces a set of variants for different processing, memory, and energy budgets.
4. **Evaluation protocol for cloud infrastructure:** texture quality is linked to instance startup time, session density, I/O operations, caching, and energy consumption.

Compared with procedural material generation [2] and graph optimization [3], the project adds shader program verification and an infrastructure objective function. Compared with neural compression [1], the result is an explicit procedural program or hybrid description that can be profiled and modified.

5. Proposed System Architecture

5.1 Input Preparation

Input images are normalized by resolution and color space and then analyzed to identify periodicity, frequency content, edges, regions, dominant colors, and multiscale structures. For materials, base-color, normal, roughness, and other property maps are processed jointly when available.

5.2 Visual Encoder

The encoder produces a set of features describing local and global patterns. Its output does not need to reconstruct the image directly; its role is to guide the selection of procedural operators, initial parameters, and region partitioning.

5.3 Procedural Graph Generator

The generator selects operators from a controlled library and connects them into a directed graph. A constrained grammar prevents invalid combinations, limits complexity, and facilitates export to multiple shader languages. Candidates are ranked using an objective function that combines perceptual quality and cost.

5.4 Parameter Optimizer

Frequency, scale, amplitude, color, threshold, and blending parameters are iteratively adjusted to the target image. Reconstruction quality, operation count, code size, compilation time, and execution time are recorded for each candidate.

5.5 Hybrid Residual Layer

When the procedural graph cannot reproduce a particular detail economically, the difference is stored as a small residual texture or a set of local patches. This enables a controlled trade-off between the purity of the procedural approach and the robustness of a conventional representation.

5.6 Shader Code Generator and Verifier

The final graph is translated into the target *shader* language. The verifier checks syntax, numerical stability, supported instructions, resource limits, and result consistency across the selected platforms.

6. Research Methodology

6.1 Dataset

The planned dataset comprises **1,200 samples** in four balanced classes: 300 stochastic surfaces, 300 periodic patterns, 300 natural materials, and 300 semantically complex scenes. Each sample is prepared at resolutions of 512 × 512, 1,024 × 1,024, and, where the source permits, 2,048 × 2,048 pixels. The split is 70% for development and training, 15% for validation, and 15% for final testing, with grouping by source material to prevent variants of the same surface from appearing in different sets. The source, license, resolution, class, color space, and available PBR maps are recorded. If the licensed dataset does not reach the planned scale, the actual number and deviation will be reported explicitly.

6.2 Baseline Methods

The baseline methods are: (1) an uncompressed reference, (2) BC7 for color and BC5 for two-channel maps, (3) AMD Compressonator with documented settings [9], (4) random-access neural compression when a reproducible implementation is available [1], and (5) a manually authored procedural graph for a stratified subset of at least 40 samples. All methods produce the same target resolution and are evaluated on the same images and hardware.

6.3 Objective Function

For candidate x, the normalized objective function is defined as: $J(x) = \alpha D(x) + \beta S(x) + \gamma T(x) + \delta M(x) + \varepsilon E(x)$, where D is perceptual distortion, S is the size of code, parameters, and residuals, T is median GPU time, M is peak memory and I/O, and E is energy per generation or representative usage cycle. Each component is divided by the value of the corresponding baseline method. The primary experiment uses equal weights of 0.20; the sensitivity analysis uses the "quality," "memory," and "cloud infrastructure cost" profiles. In addition to ranking by J, the Pareto front is reported so that the result does not depend solely on the selected weights.

6.4 Metrics

Dimension	Metrics	Purpose
Visual quality	PSNR, SSIM, perceptual metric, and user assessment	Measure fidelity and subjective acceptability
Compactness	Size of code, parameters, and residuals	Compare storage and transfer footprints
Execution	GPU/CPU execution time, instruction count, and compilation time	Estimate generation and rendering cost
Resources	VRAM/RAM, I/O operations, and energy consumption	Estimate system load
Cloud impact	Instance startup time, instance density, cache hit rate, and inter-node network traffic	Estimate infrastructure benefits

6.5 Hardware and Software Environment

In the planned evaluation, primary measurements will be performed on one predefined workstation with a discrete GPU supporting the target graphics API specification, at least 8 GB of VRAM, at least 16 GB of system memory, and SSD storage. Before the experiment begins, the exact CPU and GPU models, memory capacity, operating system, driver version, graphics API, *shader* compiler, and source-code version identifier of the implementation will be recorded in the paper. At least one additional GPU from another generation or manufacturer will be used to assess portability. Results from different devices will not be combined into a single average.

6.6 Experimental Procedure

1. Lock the test set, configuration, and random *seed* values before the final evaluation.
2. For each sample, generate at least five candidates per representation and retain the best validation candidate without inspecting the test metrics.
3. Perform ten warm-up runs and at least 100 measured rendering runs; report the median and a 95% confidence interval obtained by bootstrap resampling.
4. Measure cold instance startup, repeated use, cache hits/misses, peak memory, storage reads, and energy where the hardware supports reliable telemetry.

5. Construct a Pareto set based on perceptual distortion, size, execution time, and energy.
6. Conduct an ablation analysis without the residual layer, without the infrastructure terms of the objective function, and without each major group of procedural operators.
7. For the user assessment, use at least 24 participants, randomized order, and blinded pairwise comparisons; report the recruitment method and participant consent.

6.7 Statistical Analysis and Reproducibility

For each metric, report the median, interquartile range, and 95% confidence interval. Paired comparisons on the same textures use the Wilcoxon test with Holm correction for multiple comparisons, a significance threshold of 0.05, and an effect size. Configurations, seed values, measurement scripts, tool versions, and the distributable portion of the dataset are published.

6.8 Success Criteria

The primary criterion is for the procedural or hybrid representation on the locked test set to reduce total asset size by at least 50% relative to the corresponding BC format, with SSIM of at least 0.95 and no increase in median GPU time greater than 10%. For the cloud scenario, an additional criterion is at least 20% fewer bytes read or replicated without reducing the number of supported sessions per GPU by more than 5%. Secondary success consists of a statistically significant improvement in function J and at least one texture class for which the proposed approach lies on the Pareto front. These thresholds are predefined project targets, not claims of achieved results.

7. Application to Cloud Gaming

Cloud gaming centralizes game execution but does not eliminate computing cost or low-latency requirements [7]. Additional shader operations may reduce the number of concurrent sessions per accelerator or increase energy consumption. At the same time, compact procedural assets can reduce the size of game installation images, distribution time across regions, startup I/O, and the amount of data that must be cached.

7.1 Potential Benefits

- smaller game images and faster version distribution across data centers;
- reduced texture reads from storage during instance startup;
- better reuse of procedural programs and parameters;

- adaptive quality selection according to the available GPU budget;
- centralized profiling and automatic optimization of *shader* program variants.

7.2 Important Limitation

Procedural textures do not directly reduce the video stream sent to the end user because cloud games are generally rendered and video-encoded in the data center. Their principal network contribution concerns asset distribution, replication, and loading within the infrastructure. Any effect on outbound video bitrate can only be indirect and must be tested separately.

7.3 Adaptive Policy

The most practical project outcome may be a controller that selects one of three options for each asset: a conventionally compressed texture, a fully procedural program, or a hybrid representation. The decision is based on current GPU load, memory pressure, cache availability, usage frequency, and energy budget.

8. Broader Implications for Data Center Infrastructure

A separate section is justified because the infrastructure consequences differ from the cloud gaming scenario itself. The project results may contribute to systems in which large visual assets are frequently distributed or reconstructed, but the conclusions cannot automatically be generalized to all data types.

8.1 Storage, Replication, and Caching

Compact programs and parameters may reduce the amount of data replicated across regions and stored in multiple cache tiers. Conversely, generated outputs may require temporary space, so the entire asset lifecycle should be measured, including versioning and cache invalidation.

8.2 Computational Task Scheduling

An orchestrator can pre-generate frequently used textures during periods of lower load, while creating less common variants on demand. A predictive model can decide on which node and at what time generation is worthwhile, subject to latency constraints and accelerator availability.

8.3 Energy and Carbon Efficiency

Reduced storage alone is insufficient to support a sustainability claim. The energy required for generation, regeneration, memory access, network transfer, and data retention must be

measured. Rising data center energy use makes this type of system-level measurement relevant, particularly as accelerator workloads increase [8]. Only a full lifecycle analysis can show whether the additional computing energy is offset by savings in other components.

8.4 Applications Beyond Gaming

Potential domains include virtual environments, digital twins, simulations, spatial visualization, and synthetic data generation systems. Each domain requires separate validation of quality, determinism, security, and reproducibility.

8.5 Future Directions: Photonic and Hybrid Architectures

As a long-term research hypothesis, procedural representation may gain additional importance in future photonic and hybrid photonic-electronic data centers. Photonic accelerators offer high parallelism and bandwidth for certain classes of operations, but their system-level performance may be limited by data movement among electronic memory, converters, and optical computing units. The cost of input, output, and optoelectronic conversion can diminish the acceleration achieved by optical computation itself [10].

In such an environment, a compact procedural program and a small parameter set could replace the transfer of complete texture maps. The program would be transferred to a node or accelerator, while visual content would be reconstructed locally and on demand. This could potentially reduce the amount of data transferred between memory and the photonic accelerator, the number of repeated optoelectronic conversions, and pressure on electronic storage. This effect is not a demonstrated project benefit but a hypothesis dependent on memory architecture, reconstruction location, caching capabilities, and the energy balance between computation and conversion. Research on memory located close to photonic computing units indicates that reducing data movement and conversion is a relevant design objective [11].

Testing this hypothesis requires a separate system model or future experiment. Such a study should model the bandwidth and energy of electronic memory, optical interconnects, digital-to-analog and analog-to-digital conversion (DAC/ADC), electro-optic converters, and local procedural content reconstruction. The comparison would cover the transfer of conventional textures, neural latent representations, and procedural code under equal visual-quality requirements. Because of this additional scope, photonic architecture remains a future direction, while AI procedural textures, cloud gaming, and measurable infrastructure costs are the central focus of this work.

9. Risks and Limitations

- **Complex content:** photographs containing objects, text, or unique details may be poor candidates for a purely procedural description.
- **Execution cost:** a smaller file may result in higher total computing and energy costs.
- **Stability:** the generated shader must operate consistently across different drivers and architectures.
- **Compilation time:** a large number of shader variants may slow startup or increase cache complexity.
- **Data and licenses:** the training dataset must have clear provenance and permission for use.
- **Generalization:** results for textures do not demonstrate effectiveness for models, animations, or audio.
- **Security:** automatically generated code requires isolation, validation, and resource limits.

10. Ethical, Legal, and Environmental Considerations

10.1 Data, Copyright, and Licenses

The dataset must contain only photographs and materials for which documented permission exists for research processing, model training, and publication of derived results. For each source, records are maintained concerning the license, authorship, redistribution restrictions, and citation method. If the dataset contains identifiable persons, private spaces, or sensitive metadata, these elements are removed or processed only with an appropriate legal basis and consent.

10.2 Validation and Security of AI-Generated Code

Generated *shader* code must not be executed without automated checks of syntax, resource constraints, and behavior in an isolated environment. The system should limit execution duration, memory access, program size, and the permitted operator set. All results must be labeled as AI-system output, while failed, unstable, and manually corrected variants must be recorded for reproducibility.

10.3 Transparency and Reproducibility

The study should clearly distinguish planned thresholds from achieved results, as well as measured values from estimates and simulations. Model and tool versions, configurations, random *seed* values, candidate-selection rules, and statistical analysis methods are published. When data or code cannot be made public, the reasons are stated and descriptive protocols are provided to enable independent verification to the greatest extent possible.

10.4 Environmental Assessment

Claims of energy or carbon benefits are based on measurement of the entire relevant lifecycle: model preparation and training, representation generation, compilation, rendering, storage, and network transfer. A reduction in file size alone is not evidence of sustainability. The project should report energy per processed sample and per representative usage cycle and limit its conclusions to the tested hardware and infrastructure conditions.

11. Research and Development Plan

Phase	Activities	Outcome
1. Specification and literature review	Define the grammar, scenarios, metrics, and baseline methods	Research protocol
2. Data and prototype	Prepare the dataset and implement the procedural library	Reproducible baseline system
3. AI synthesis	Develop the encoder, graph generator, and optimizer	Candidate-generating model
4. *Shader* and hybrid layer	Code export, verification, and residual representation	Executable prototype
5. Evaluation	Compare quality, size, speed, energy, and cloud infrastructure metrics	Experimental results
6. Demonstrator	Integrate the adaptive controller and user interface	R&D demonstrator and final documentation

12. Expected Results

Because this document is a research and development project proposal, the stated values are targets that will be tested only after implementation. The expected outcome is a functional prototype that produces a procedural or hybrid description, an executable shader, and a report on quality-resource trade-offs from an input texture. The planned scientific contribution includes the formulation of multi-objective program synthesis, a comparison methodology, and an analysis of the conditions under which procedural representation is beneficial for cloud infrastructure. Percentage savings and statistical conclusions will not be presented as achieved until they are derived from measurements.

13. Conclusion

This project proposal connects AI, procedural graphics, and infrastructure resource management. Its strongest form is not based on the assumption that computing power will become free, but on adaptive representation selection according to the actual state of the

system. Cloud gaming provides a relevant and measurable application case, while a separate analysis of data centers enables broader conclusions without losing the focus of the work. The key to success will be demonstrating that, in certain scenarios, savings in storage, transfer, and I/O operations exceed the additional computing and energy costs.

14. References

[1] K. Vaidyanathan, M. Salvi, B. Wronski, T. Akenine-Möller, P. Ebelin, and A. Lefohn, "Random-Access Neural Compression of Material Textures," in *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.

[2] Y. Hu, P. Guerrero, M. Hašan, H. Rushmeier, and V. Deschaintre, "Generating Procedural Materials from Text or Image Prompts," in *ACM SIGGRAPH 2023 Conference Proceedings*, 2023, doi: 10.1145/3588432.3591520.

[3] Y. Hu, P. Guerrero, M. Hašan, H. Rushmeier, and V. Deschaintre, "Node Graph Optimization Using Differentiable Proxies," in *ACM SIGGRAPH 2022 Conference Proceedings*, 2022, doi: 10.1145/3528233.3530733.

[4] A. Lagae, S. Lefebvre, G. Drettakis, and P. Dutré, "Procedural Noise using Sparse Gabor Convolution," *ACM Transactions on Graphics*, vol. 28, no. 3, art. 54, 2009, doi: 10.1145/1531326.1531360.

[5] R. L. Cook and T. DeRose, "Wavelet Noise," *ACM Transactions on Graphics*, vol. 24, no. 3, pp. 803–811, 2005, doi: 10.1145/1073204.1073264.

[6] J. C. Hart, "Perlin Noise Pixel Shaders," in *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Workshop on Graphics Hardware*, 2001, doi: 10.1145/383507.383531.

[7] W. Cai, R. Shea, C.-Y. Huang, K.-T. Chen, J. Liu, V. C. M. Leung, and C.-H. Hsu, "A Survey on Cloud Gaming: Future of Computer Games," *IEEE Access*, 2016, IEEE document 7536162.

[8] A. Shehabi et al., *2024 United States Data Center Energy Usage Report*. Berkeley, CA, USA: Lawrence Berkeley National Laboratory, 2024, doi: 10.71468/P1WC7Q.

[9] AMD GPUOpen, "AMD Compressonator," 2024. [Online]. Available: https://gpuopen.com/compressonator/. [Accessed: October 19, 2026].

[10] J. T. Meech, V. Tsoutsouras, and P. Stanley-Marbell, "The Data Movement Bottleneck: Theoretical Shortcomings of Analog Optical Fourier Transform and Convolution Computing Accelerators," arXiv:2308.01719, 2023.

[11] S. Lam et al., "Neuromorphic Photonic Computing with an Electro-Optic Analog Memory," *Nature Communications*, vol. 17, art. 2472, 2026, doi: 10.1038/s41467-026-69084-x.